Minimum, Maximum, and Mixture

```
# install empiricaldist if necessary

try:
    import empiricaldist
except ImportError:
    !pip install empiricaldist
    import empiricaldist
```

```
# Get utils.py

from os.path import basename, exists

def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve
        local, _ = urlretrieve(url, filename)
        print('Downloaded ' + local)

download('https://github.com/AllenDowney/ThinkBayes2/raw/master/soln/utils.py')
```

```
from utils import set_pyplot_params
set_pyplot_params()
```

In the previous chapter we computed distributions of sums. In this chapter, we'll compute distributions of minimums and maximums, and use them to solve both forward and inverse problems.

Then we'll look at distributions that are mixtures of other distributions, which will turn out to be particularly useful for making predictions.

But we'll start with a powerful tool for working with distributions, the cumulative distribution function.

Cumulative Distribution Functions

So far we have been using probability mass functions to represent distributions. A useful alternative is the **cumulative distribution function**, or CDF.

As an example, I'll use the posterior distribution from the Euro problem, which we computed in <<_BayesianEstimation>>.

Here's the uniform prior we started with.

```
import numpy as np
from empiricaldist import Pmf

hypos = np.linspace(0, 1, 101)
pmf = Pmf(1, hypos)
data = 140, 250
```

And here's the update.

```
from scipy.stats import binom

def update_binomial(pmf, data):
    """Update pmf using the binomial distribution."""
    k, n = data
    xs = pmf.qs
    likelihood = binom.pmf(k, n, xs)
    pmf *= likelihood
    pmf.normalize()
```

```
update_binomial(pmf, data)
```

The CDF is the cumulative sum of the PMF, so we can compute it like this:

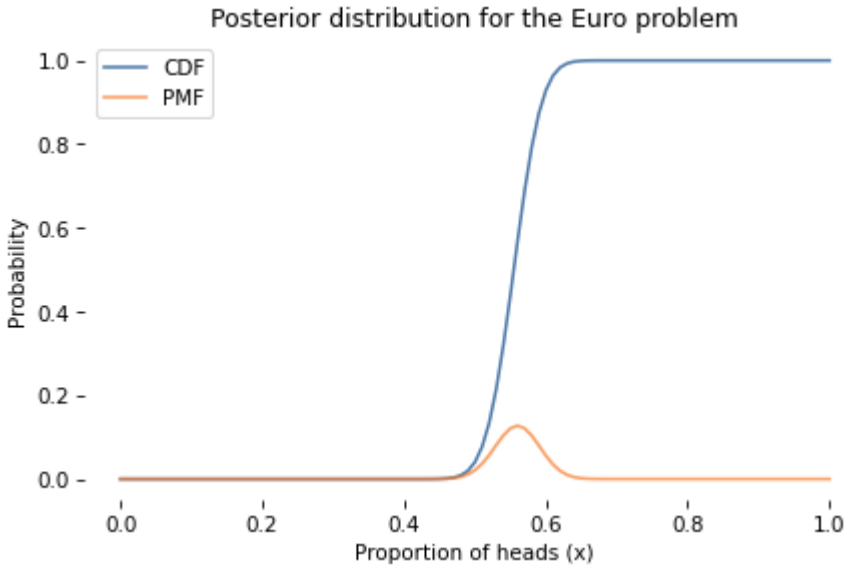
```
cumulative = pmf.cumsum()
```

Here's what it looks like, along with the PMF.

```
from utils import decorate

def decorate_euro(title):
    decorate(xlabel='Proportion of heads (x)',
             ylabel='Probability',
             title=title)
```

```
cumulative.plot(label='CDF')
pmf.plot(label='PMF')
decorate_euro(title='Posterior distribution for the Euro problem')
```



The range of the CDF is always from 0 to 1, in contrast with the PMF, where the maximum can be any probability.

The result from `cumsum` is a Pandas `Series`, so we can use the bracket operator to select an element:

```
cumulative[0.61]
```

```
np.float64(0.9638303193984255)
```

The result is about 0.96, which means that the total probability of all quantities less than or equal to 0.61 is 96%.

To go the other way --- to look up a probability and get the corresponding quantile --- we can use interpolation:

```
from scipy.interpolate import interp1d

ps = cumulative.values
qs = cumulative.index

interp = interp1d(ps, qs)
interp(0.96)
```

```
array(0.60890171)
```

The result is about 0.61, so that confirms that the 96th percentile of this distribution is 0.61.

`empiricaldist` provides a class called `Cdf` that represents a cumulative distribution function. Given a `Pmf`, you can compute a `Cdf` like this:

```
cdf = pmf.make_cdf()
```

`make_cdf` uses `np.cumsum` to compute the cumulative sum of the probabilities.

You can use brackets to select an element from a `Cdf`:

```
cdf[0.61]
```

```
np.float64(0.9638303193984255)
```

But if you look up a quantity that's not in the distribution, you get a `KeyError`.

```
try:
    cdf[0.615]
except KeyError as e:
    print(repr(e))
```

```
KeyError(0.615)
```

To avoid this problem, you can call a `Cdf` as a function, using parentheses. If the argument does not appear in the `Cdf`, it interpolates between quantities.

```
cdf(0.615)
```

```
array(0.96383032)
```

Going the other way, you can use `quantile` to look up a cumulative probability and get the corresponding quantity:

```
cdf.quantile(0.9638303)
```

```
array(0.61)
```

`Cdf` also provides `credible_interval`, which computes a credible interval that contains the given probability:

```
cdf.credible_interval(0.9)
```

```
array([0.51, 0.61])
```

CDFs and PMFs are equivalent in the sense that they contain the same information about the distribution, and you can always convert from one to the other. Given a `Cdf`, you can get the equivalent `Pmf` like this:

```
pmf = cdf.make_pmf()
```

`make_pmf` uses `np.diff` to compute differences between consecutive cumulative probabilities.

One reason `Cdf` objects are useful is that they compute quantiles efficiently. Another is that they make it easy to compute the distribution of a maximum or minimum, as we'll see in the next section.

Best Three of Four

In *Dungeons & Dragons*, each character has six attributes: strength, intelligence, wisdom, dexterity, constitution, and charisma.

To generate a new character, players roll four 6-sided dice for each attribute and add up the best three. For example, if I roll for strength and get 1, 2, 3, 4 on the dice, my character's strength would be the sum of 2, 3, and 4, which is 9.

As an exercise, let's figure out the distribution of these attributes. Then, for each character, we'll figure out the distribution of their best attribute.

I'll import two functions from the previous chapter: `make_die`, which makes a `Pmf` that represents the outcome of rolling a die, and `add_dist_seq`, which takes a sequence of `Pmf` objects and computes the distribution of their sum.

Here's a `Pmf` that represents a six-sided die and a sequence with three references to it.

```
from utils import make_die

die = make_die(6)
dice = [die] * 3
```

And here's the distribution of the sum of three dice.

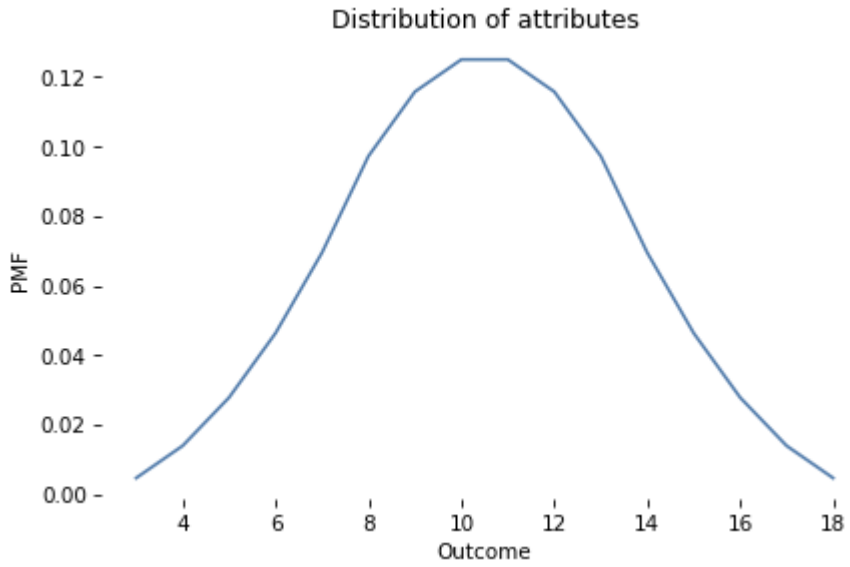
```
from utils import add_dist_seq

pmf_3d6 = add_dist_seq(dice)
```

Here's what it looks like:

```
def decorate_dice(title=''):
    decorate(xlabel='Outcome',
             ylabel='PMF',
             title=title)
```

```
pmf_3d6.plot()
decorate_dice('Distribution of attributes')
```



If we roll four dice and add up the best three, computing the distribution of the sum is a bit more complicated. I'll estimate the distribution by simulating 10,000 rolls.

First I'll create an array of random values from 1 to 6, with 10,000 rows and 4 columns:

```
n = 10000
a = np.random.randint(1, 7, size=(n, 4))
```

To find the best three outcomes in each row, I'll use `sort` with `axis=1`, which sorts the rows in ascending order.

```
a.sort(axis=1)
```

Finally, I'll select the last three columns and add them up.

```
t = a[:, 1:].sum(axis=1)
```

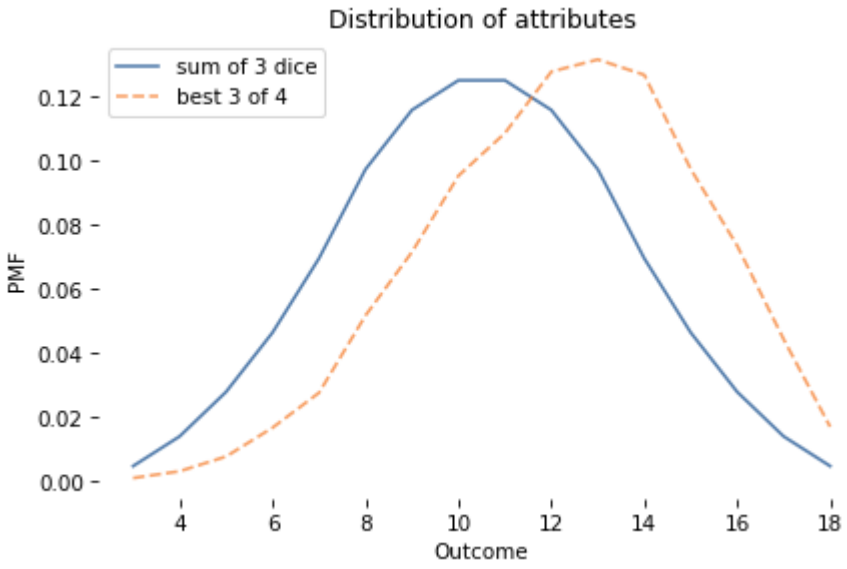
Now `t` is an array with a single column and 10,000 rows. We can compute the PMF of the values in `t` like this:

```
pmf_best3 = Pmf.from_seq(t)
```

The following figure shows the distribution of the sum of three dice, `pmf_3d6`, and the distribution of the best three out of four, `pmf_best3`.

```
pmf_3d6.plot(label='sum of 3 dice')
pmf_best3.plot(label='best 3 of 4', ls='--')

decorate_dice('Distribution of attributes')
```



As you might expect, choosing the best three out of four tends to yield higher values.

Next we'll find the distribution for the maximum of six attributes, each the sum of the best three of four dice.

Maximum

To compute the distribution of a maximum or minimum, we can make good use of the cumulative distribution function. First, I'll compute the `Cdf` of the best three of four distribution:

```
cdf_best3 = pmf_best3.make_cdf()
```

Recall that `Cdf(x)` is the sum of probabilities for quantities less than or equal to `x`. Equivalently, it is the probability that a random value chosen from the distribution is less than or equal to `x`.

Now suppose I draw 6 values from this distribution. The probability that all 6 of them are less than or equal to `x` is `Cdf(x)` raised to the 6th power, which we can compute like this:

```
cdf_best3**6
```

```
3     5.314410e-19
4     3.518744e-15
5     2.313061e-12
6     4.923092e-10
7     2.954260e-08
8     1.517640e-06
9     3.245553e-05
10    4.213089e-04
11    3.121949e-03
12    1.759629e-02
13    6.969152e-02
14    2.055161e-01
15    4.203417e-01
16    6.850402e-01
17    9.027888e-01
18    1.000000e+00
Name: , dtype: float64
```

If all 6 values are less than or equal to `x`, that means that their maximum is less than or equal to `x`. So the result is the CDF of their maximum. We can convert it to a `Cdf` object, like this:

```
from empiricaldist import Cdf

cdf_max6 = Cdf(cdf_best3**6)
```

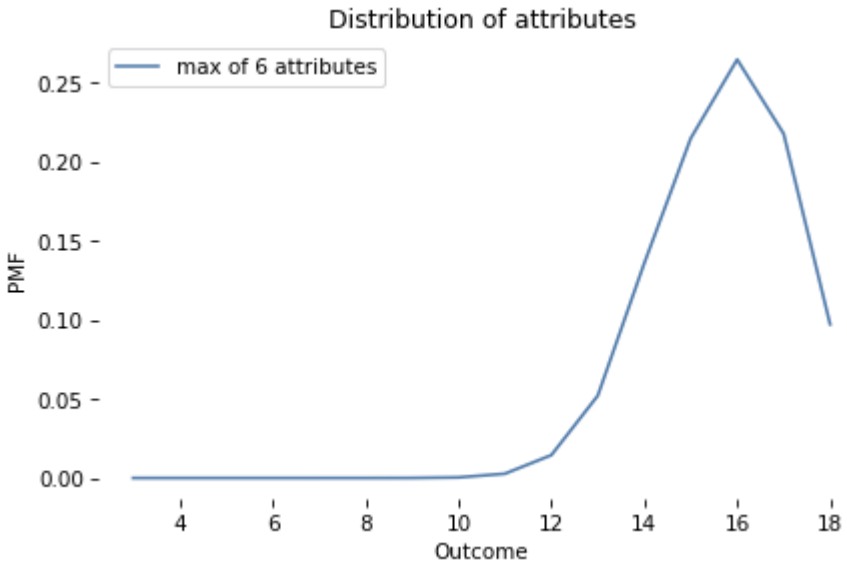
And compute the equivalent `Pmf` like this:

```
pmf_max6 = cdf_max6.make_pmf()
```

The following figure shows the result.

```
pmf_max6.plot(label='max of 6 attributes')

decorate_dice('Distribution of attributes')
```



Most characters have at least one attribute greater than 12; almost 10% of them have an 18.

The following figure shows the CDFs for the three distributions we have computed.

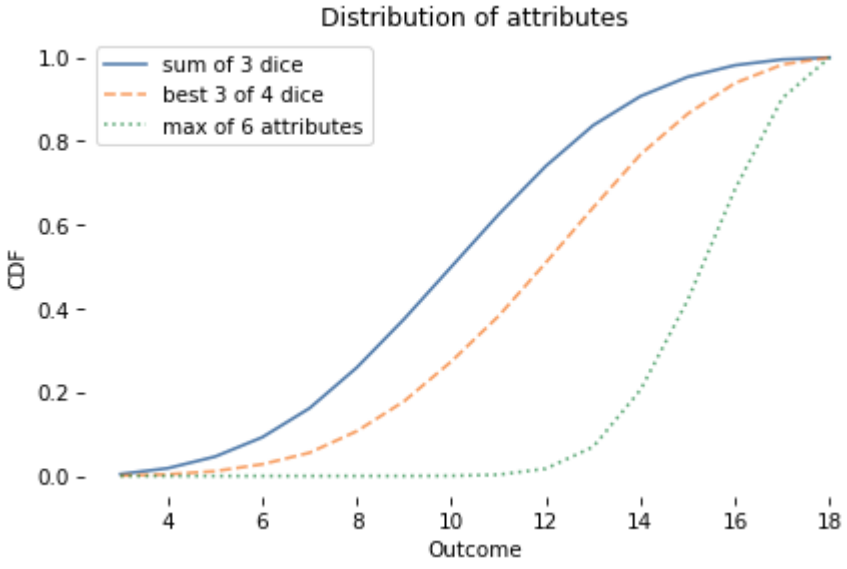
```
import matplotlib.pyplot as plt

cdf_3d6 = pmf_3d6.make_cdf()
cdf_3d6.plot(label='sum of 3 dice')

cdf_best3 = pmf_best3.make_cdf()
cdf_best3.plot(label='best 3 of 4 dice', ls='--')

cdf_max6.plot(label='max of 6 attributes', ls=':')

decorate_dice('Distribution of attributes')
plt.ylabel('CDF');
```



`Cdf` provides `max_dist`, which does the same computation, so we can also compute the `Cdf` of the maximum like this:

```
cdf_max_dist6 = cdf_best3.max_dist(6)
```

And we can confirm that the differences are small.

```
np.allclose(cdf_max_dist6, cdf_max6)
```

```
True
```

In the next section we'll find the distribution of the minimum. The process is similar, but a little more complicated. See if you can figure it out before you go on.

Minimum

In the previous section we computed the distribution of a character's best attribute. Now let's compute the distribution of the worst.

To compute the distribution of the minimum, we'll use the **complementary CDF**, which we can compute like this:

```
prob_gt = 1 - cdf_best3
```

As the variable name suggests, the complementary CDF is the probability that a value from the distribution is greater than `x`. If we draw 6 values from the distribution, the probability that all 6 exceed `x` is:

```
prob_gt6 = prob_gt**6
```

If all 6 exceed `x`, that means their minimum exceeds `x`, so `prob_gt6` is the complementary CDF of the minimum. And that means we can compute the CDF of the minimum like this:

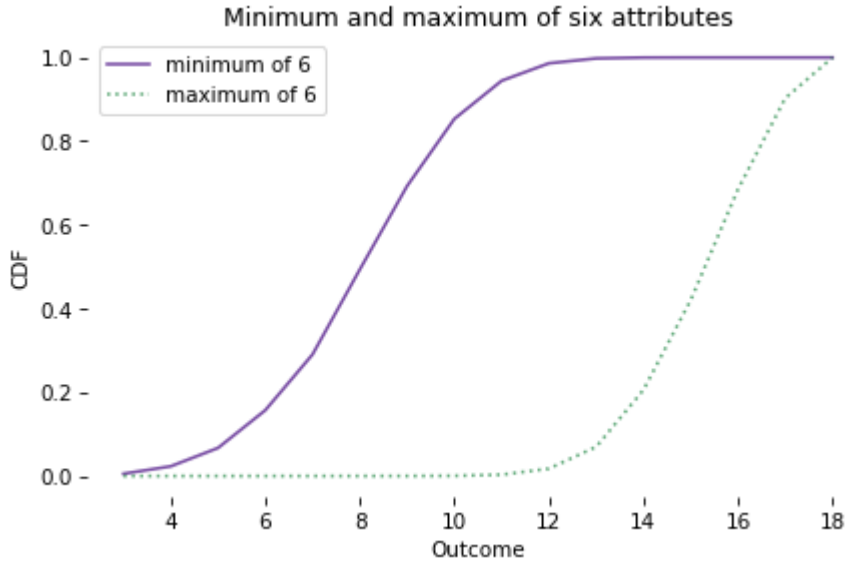
```
prob_le6 = 1 - prob_gt6
```

The result is a Pandas `Series` that represents the CDF of the minimum of six attributes. We can put those values in a `Cdf` object like this:

```
cdf_min6 = Cdf(prob_le6)
```

Here's what it looks like, along with the distribution of the maximum.

```
cdf_min6.plot(color='C4', label='minimum of 6')
cdf_max6.plot(color='C2', label='maximum of 6', ls=':')
decorate_dice('Minimum and maximum of six attributes')
plt.ylabel('CDF');
```



`Cdf` provides `min_dist`, which does the same computation, so we can also compute the `Cdf` of the minimum like this:

```
cdf_min_dist6 = cdf_best3.min_dist(6)
```

And we can confirm that the differences are small.

```
np.allclose(cdf_min_dist6, cdf_min6)
```

```
True
```

In the exercises at the end of this chapter, you'll use distributions of the minimum and maximum to do Bayesian inference. But first we'll see what happens when we mix distributions.

Mixture

In this section I'll show how we can compute a distribution which is a mixture of other distributions. I'll explain what that means with some simple examples; then, more usefully, we'll see how these mixtures are used to make predictions.

Here's another example inspired by *Dungeons & Dragons*:

- Suppose your character is armed with a dagger in one hand and a short sword in the other.
- During each round, you attack a monster with one of your two weapons, chosen at random.
- The dagger causes one 4-sided die of damage; the short sword causes one 6-sided die of damage.

What is the distribution of damage you inflict in each round?

To answer this question, I'll make a `Pmf` to represent the 4-sided and 6-sided dice:

```
d4 = make_die(4)
d6 = make_die(6)
```

Now, let's compute the probability you inflict 1 point of damage.

- If you attacked with the dagger, it's 1/4.
- If you attacked with the short sword, it's 1/6.

Because the probability of choosing either weapon is 1/2, the total probability is the average:

```
prob_1 = (d4(1) + d6(1)) / 2
prob_1
```

```
np.float64(0.20833333333333331)
```

For the outcomes 2, 3, and 4, the probability is the same, but for 5 and 6 it's different, because those outcomes are impossible with the 4-sided die.

```
prob_6 = (d4(6) + d6(6)) / 2
prob_6
```

```
np.float64(0.08333333333333333)
```

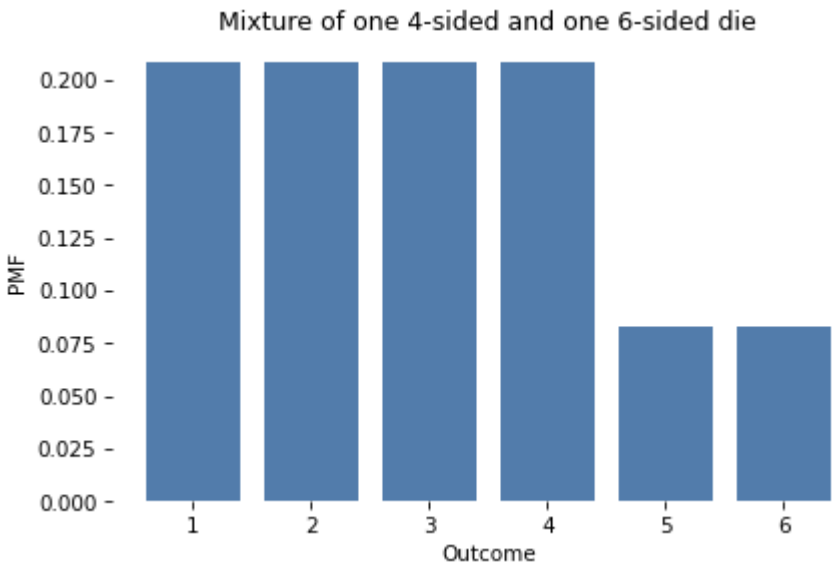
To compute the distribution of the mixture, we could loop through the possible outcomes and compute their probabilities.

But we can do the same computation using the `+` operator:

```
mix1 = (d4 + d6) / 2
```

Here's what the mixture of these distributions looks like.

```
mix1.bar(alpha=0.7)
decorate_dice('Mixture of one 4-sided and one 6-sided die')
```



Now suppose you are fighting three monsters:

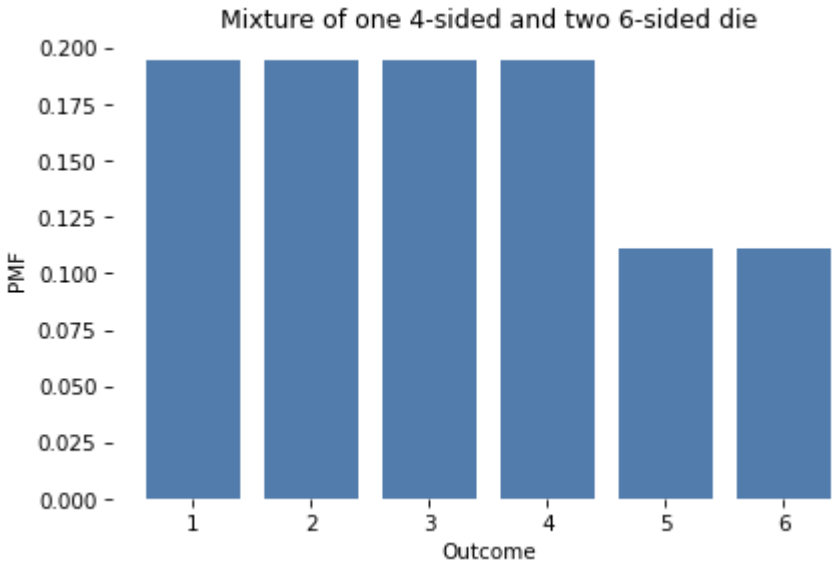
- One has a club, which causes one 4-sided die of damage.
- One has a mace, which causes one 6-sided die.
- And one has a quarterstaff, which also causes one 6-sided die.

Because the melee is disorganized, you are attacked by one of these monsters each round, chosen at random. To find the distribution of the damage they inflict, we can compute a weighted average of the distributions, like this:

```
mix2 = (d4 + 2*d6) / 3
```

This distribution is a mixture of one 4-sided die and two 6-sided dice. Here's what it looks like.

```
mix2.bar(alpha=0.7)
decorate_dice('Mixture of one 4-sided and two 6-sided die')
```



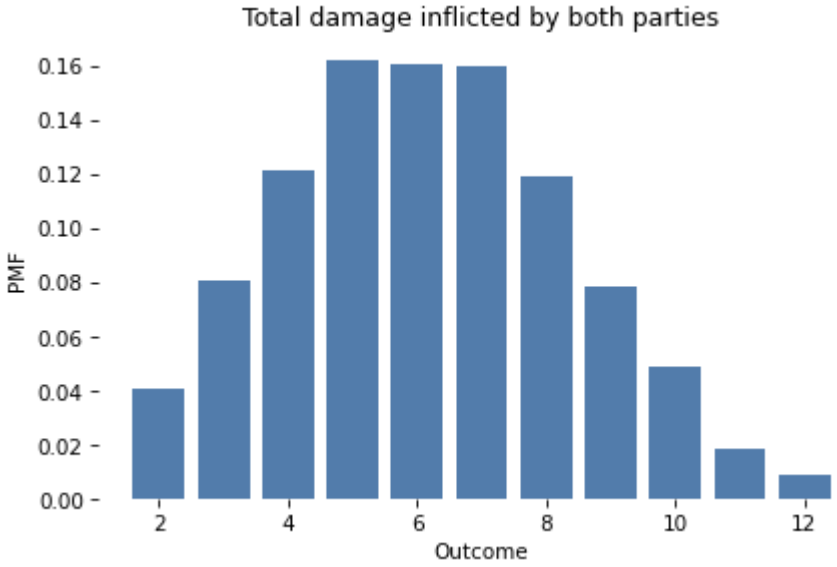
In this section we used the `+` operator, which adds the probabilities in the distributions, not to be confused with `Pmf.add_dist`, which computes the distribution of the sum of the distributions.

To demonstrate the difference, I'll use `Pmf.add_dist` to compute the distribution of the total damage done per round, which is the sum of the two mixtures:

```
total_damage = Pmf.add_dist(mix1, mix2)
```

And here's what it looks like.

```
total_damage.bar(alpha=0.7)
decorate_dice('Total damage inflicted by both parties')
```



General Mixtures

In the previous section we computed mixtures in an *ad hoc* way. Now we'll see a more general solution. In future chapters, we'll use this solution to generate predictions for real-world problems, not just role-playing games. But if you'll bear with me, we'll continue the previous example for one more section.

Suppose three more monsters join the combat, each of them with a battle axe that causes one 8-sided die of damage. Still, only one monster attacks per round, chosen at random, so the damage they inflict is a mixture of:

- One 4-sided die,
- Two 6-sided dice, and
- Three 8-sided dice.

I'll use a `Pmf` to represent a randomly chosen monster:

```
hypos = [4,6,8]
counts = [1,2,3]
pmf_dice = Pmf(counts, hypos)
pmf_dice.normalize()
pmf_dice
```

	probs
4	0.166667
6	0.333333
8	0.500000

This distribution represents the number of sides on the die we'll roll and the probability of rolling each one. For example, one of the six monsters has a dagger, so the probability is 1/6 that we roll a 4-sided die.

Next I'll make a sequence of `Pmf` objects to represent the dice:

```
dice = [make_die(sides) for sides in hypos]
```

To compute the distribution of the mixture, I'll compute the weighted average of the dice, using the probabilities in `pmf_dice` as the weights.

To express this computation concisely, it is convenient to put the distributions into a Pandas `DataFrame`:

```
import pandas as pd

pd.DataFrame(dice)
```

	1	2	3	4	5	6	
0	0.250000	0.250000	0.250000	0.250000	NaN	NaN	
1	0.166667	0.166667	0.166667	0.166667	0.166667	0.166667	
2	0.125000	0.125000	0.125000	0.125000	0.125000	0.125000	0

The result is a `DataFrame` with one row for each distribution and one column for each possible outcome. Not all rows are the same length, so Pandas fills the extra spaces with the special value `NaN`, which stands for "not a number". We can use `fillna` to replace the `NaN` values with 0.

pd.DataFrame(dice).fillna(0)

The next step is to multiply each row by the probabilities in `pmf_dice`, which turns out to be easier if we transpose the matrix so the distributions run down the columns rather than across the rows:

```
df = pd.DataFrame(dice).fillna(0).transpose()
```

```
df
```

	0	1	2
1	0.25	0.166667	0.125
2	0.25	0.166667	0.125
3	0.25	0.166667	0.125
4	0.25	0.166667	0.125
5	0.00	0.166667	0.125
6	0.00	0.166667	0.125
7	0.00	0.000000	0.125
8	0.00	0.000000	0.125

Now we can multiply by the probabilities in `pmf_dice`:

```
df *= pmf_dice.ps
```

```
df
```

	0	1	2
1	0.041667	0.055556	0.0625
2	0.041667	0.055556	0.0625
3	0.041667	0.055556	0.0625
4	0.041667	0.055556	0.0625
5	0.000000	0.055556	0.0625
6	0.000000	0.055556	0.0625
7	0.000000	0.000000	0.0625
8	0.000000	0.000000	0.0625

And add up the weighted distributions:

```
df.sum(axis=1)
```

```
1    0.159722
2    0.159722
3    0.159722
4    0.159722
5    0.118056
6    0.118056
7    0.062500
8    0.062500
dtype: float64
```

The argument `axis=1` means we want to sum across the rows. The result is a Pandas `Series`.

Putting it all together, here's a function that makes a weighted mixture of distributions.

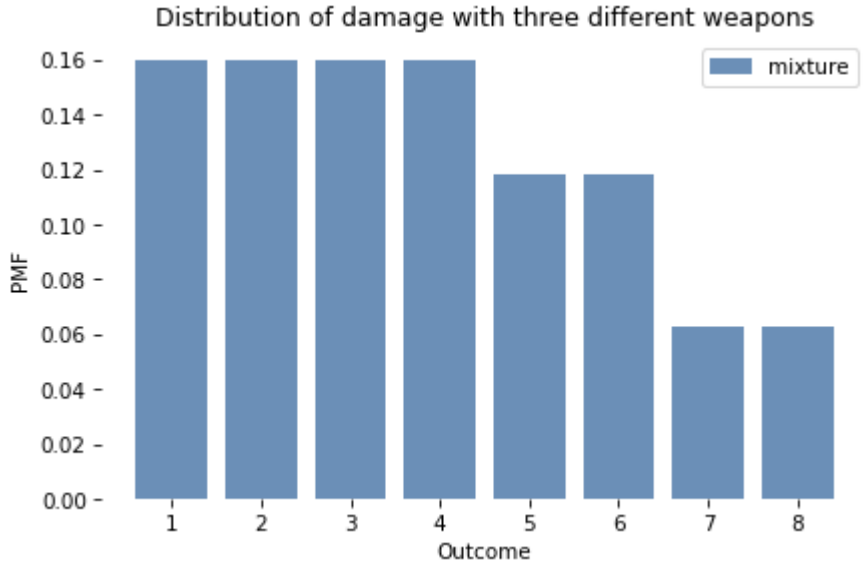
```
def make_mixture(pmf, pmf_seq):
    """Make a mixture of distributions."""
    df = pd.DataFrame(pmf_seq).fillna(0).transpose()
    df *= np.array(pmf)
    total = df.sum(axis=1)
    return Pmf(total)
```

The first parameter is a `Pmf` that maps from each hypothesis to a probability. The second parameter is a sequence of `Pmf` objects, one for each hypothesis. We can call it like this:

```
mix = make_mixture(pmf_dice, dice)
```

And here's what it looks like.

```
mix.bar(label='mixture', alpha=0.6)
decorate_dice('Distribution of damage with three different weapons')
```



In this section I used Pandas so that `make_mixture` is concise, efficient, and hopefully not too hard to understand. In the exercises at the end of the chapter, you'll have a chance to practice with mixtures, and we will use `make_mixture` again in the next chapter.

Summary

This chapter introduces the `Cdf` object, which represents the cumulative distribution function (CDF).

A `Pmf` and the corresponding `Cdf` are equivalent in the sense that they contain the same information, so you can convert from one to the other.
The primary difference between them is performance: some operations are faster and easier with a `Pmf`; others are faster with a `Cdf`.

In this chapter we used `Cdf` objects to compute distributions of maximums and minimums; these distributions are useful for inference if we are given a maximum or minimum as data. You will see some examples in the exercises, and in future chapters. We also computed mixtures of distributions, which we will use in the next chapter to make predictions.

But first you might want to work on these exercises.

Exercises

Exercise: When you generate a D&D character, instead of rolling dice, you can use the "standard array" of attributes, which is 15, 14, 13, 12, 10, and 8. Do you think you are better off using the standard array or (literally) rolling the dice?

Compare the distribution of the values in the standard array to the distribution we computed for the best three out of four:

- Which distribution has higher mean? Use the `mean` method.
- Which distribution has higher standard deviation? Use the `std` method.
- The lowest value in the standard array is 8. For each attribute, what is the probability of getting a value less than 8? If you roll the dice six times, what's the probability that at least one of your attributes is less than 8?
- The highest value in the standard array is 15. For each attribute, what is the probability of getting a value greater than 15? If you roll the dice six times, what's the probability that at least one of your attributes is greater than 15?

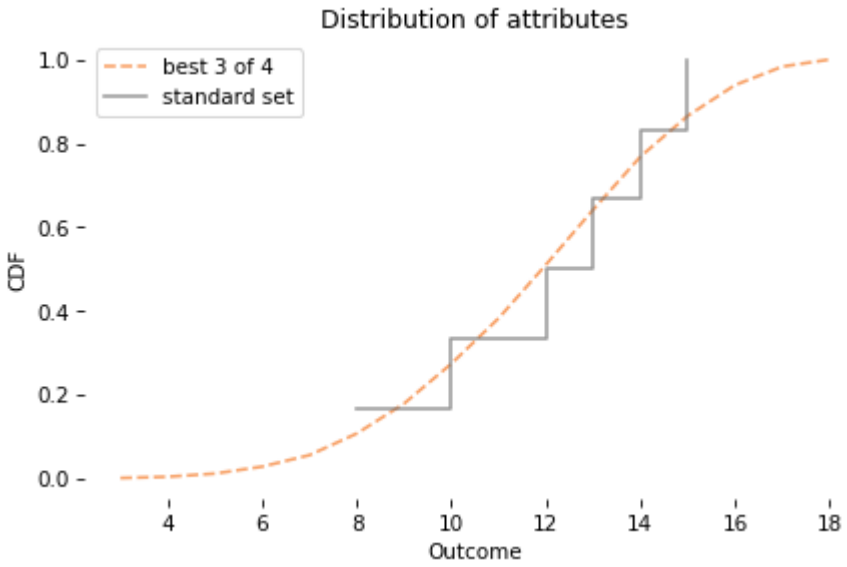
To get you started, here's a `Cdf` that represents the distribution of attributes in the standard array:

```
standard = [15,14,13,12,10,8]
cdf_standard = Cdf.from_seq(standard)
```

We can compare it to the distribution of attributes you get by rolling four dice at adding up the best three.

```
cdf_best3.plot(label='best 3 of 4', color='C1', ls='--')
cdf_standard.step(label='standard set', color='C7')

decorate_dice('Distribution of attributes')
plt.ylabel('CDF');
```



I plotted `cdf_standard` as a step function to show more clearly that it contains only a few quantities.

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Exercise: Suppose you are fighting three monsters:

- One is armed with a short sword that causes one 6-sided die of damage,
- One is armed with a battle axe that causes one 8-sided die of damage, and
- One is armed with a bastard sword that causes one 10-sided die of damage.

One of the monsters, chosen at random, attacks you and does 1 point of damage.

Which monster do you think it was? Compute the posterior probability that each monster was the attacker.

If the same monster attacks you again, what is the probability that you suffer 6 points of damage?

Hint: Compute a posterior distribution as we have done before and pass it as one of the arguments to `make_mixture`.

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

```
# Solution goes here
```

Exercise: Henri Poincaré was a French mathematician who taught at the Sorbonne around 1900. The following anecdote about him is probably fiction, but it makes an interesting probability problem.

Supposedly Poincaré suspected that his local bakery was selling loaves of bread that were lighter than the advertised weight of 1 kg, so every day for a year he bought a loaf of bread, brought it home and weighed it. At the end of the year, he plotted the distribution of his measurements and showed that it fit a normal distribution with mean 950 g and standard deviation 50 g. He brought this evidence to the bread police, who gave the baker a warning.

For the next year, Poincaré continued to weigh his bread every day. At the end of the year, he found that the average weight was 1000 g, just as it should be, but again he complained to the bread police, and this time they fined the baker.

Why? Because the shape of the new distribution was asymmetric. Unlike the normal distribution, it was skewed to the right, which is consistent with the hypothesis that the baker was still making 950 g loaves, but deliberately giving Poincaré the heavier ones.

To see whether this anecdote is plausible, let's suppose that when the baker sees Poincaré coming, he hefts `n` loaves of bread and gives Poincaré the heaviest one. How many loaves would the baker have to heft to make the average of the maximum 1000 g?

To get you started, I'll generate a year's worth of data from a normal distribution with the given parameters.

```
mean = 950
std = 50

np.random.seed(17)
sample = np.random.normal(mean, std, size=365)
```

```
# Solution goes here
```

```
# Solution goes here
```